

Object-Oriented Programming – Basic Concepts

Note: This document was translated from Hungarian to English by AI. Please verify the information against the original Hungarian source (definiciok.pdf) before relying on it.

Programming Paradigm	Programming style. Essentially it means the set of tools used for building up the program, that is, what units form the building blocks of the program. (modular programming, object-oriented programming, generic programming, aspect-oriented programming, etc.)
Abstract Data Type (ADT)	The highest level of describing a data type, in which we specify the data type so that we give no prescription whatsoever for the representation of the data and the implementation of the operations. We describe the data type preferably using mathematical concepts (sets and operations defined on them).
Abstraction	Abstracting away. With its help we can hide private implementations behind a public interface. Example: the <code>List</code> interface in the <code>java.util</code> package and the <code>ArrayList</code> and <code>LinkedList</code> classes implementing the interface. Abstraction makes it possible to handle the instances of both classes through the operations of the same <code>List</code> interface.
Object Oriented Programming	A programming paradigm that builds up programs from objects. The operation of the program essentially means the communication of objects. Its most important principles: encapsulation, inheritance, polymorphism.
Class	A class is a user-defined type, on the basis of which instances (objects) can be created. A class essentially contains data and method (operation) definitions.
Object (Instance)	It stores information (data) and performs operations on request. It has state, behavior, and is identifiable at run time.
Message	A request forwarded to an object. As a response, the object executes the requested operation.
Encapsulation	It means the enclosing of data and methods into a class. The object essentially encapsulates the state (the values of the data members) together with the behavior (the operations). Consequence: we can modify the state of the object only through its operations.
Information Hiding	The object hides its data and certain operations. This means that we do not know exactly how the data is represented inside an object, and indeed we do not even know the implementations of the operations. Hiding information serves the safety of the object, which we can only access through controlled operations.

Inheritance	A relationship defined between classes, with the help of which we can create a more specific type (descendant class) from a more general type (ancestor class). The descendant class inherits data and operations (behavior), supplements these with its own data and operations, and may override certain operations. One of the ways of code reuse. We distinguish single and multiple inheritance.
Code Reuse	The use of a given class to create new classes via inheritance, aggregation, or composition.
Dynamic (Late) Binding	The run-time assignment of the called method to the object. (E.g. C++: virtual functions, Java: instance methods) (Slow)
Substitutability	If S is a subtype of type T (we derive class S from class T), then we can substitute instances of class T with instances of class S, without endangering the correctness of our program.
Software Reuse	Function libraries. Class libraries. Frameworks.
Static (Early) Binding	The compile-time assignment of the called method to the object. (E.g. C++: non-virtual functions, Java: class methods – static methods) (Fast)
Methods Overloading	Several functions with the same name but different signatures. The actual parameters of the function call determine which function will be invoked. This is already decided at compile time (static, compile-time binding).
Methods Overriding	Within a class hierarchy, the descendant class redefines a method of the ancestor class (same name, same signature). If we access instances of the class hierarchy through a pointer or reference of the ancestor type and through it call the overridden method, then it is decided at run time exactly which method will be invoked (dynamic, run-time binding).
Abstract Class	A class that has at least one abstract operation. It defines an interface and cannot be instantiated. An abstract class leaves the implementation of its abstract operations to the descendant classes.
Concrete Class	A class that does not contain any abstract operation. It can be instantiated.
Interface (Java)	It defines a behavior. In practice it means the declaration of a set of operations. If a class implements a given interface, then its instances will have the behavior specified in the interface. It can contain only constant data members and all of its members are public.
Object Interface	It determines the set of operations that can be performed with the object.
Polymorphism	Many-formedness. A type-theoretical concept according to which a variable of an ancestor type can refer to instances of classes derived from the same common ancestor class (or implementing the same interface). Polymorphism can be static and dynamic. (a) static polymorphism: method overloading, function templates, class templates. Static, compile-time binding. (b) dynamic polymorphism: method overriding. Dynamic, run-time binding.
Constructor	The operation that initializes the object. It is called automatically. We create as many kinds of constructor for a class as the number of ways in which we make it possible to initialize the instances.
Destructor	The operation opposite to the constructor; it generally releases the resources acquired in the constructor. It is executed before the destruction of the object and is called automatically.

Coupling	The degree of dependency between components. We distinguish loosely and tightly coupled systems. In the case of loosely coupled systems, the change of some component of the system does not entail the modification of the other components.
Class (Static) Members	Static members = static data members + static method members. Static data members are data that the instances of the given class use in common (shared). Static operations are operations that work on their arguments and on the static data members of the class. These members can be used even before the creation of instances.
Aggregation	A part-whole relationship. The parts make up the whole. For example a car is an aggregation of engine, chassis, and wheels. The parts can outlive the whole.
Composition	A special aggregation, when the part belongs tightly to the whole. The parts do not outlive the whole. For example the human brain belongs tightly to the human.
Delegation	An implementation mechanism during which an object forwards (delegates) the request to another object. The delegated object will process the request. Example: Java event handling (the request is forwarded to the event listener).
Container	A type that provides the storage of objects. Besides the storage function it also provides maintenance operations.
Iterator	A type that determines a position in a collection (container, data stream). Through its operations it provides the traversal of the container, that is, the consecutive processing of the stored elements.

C++ concepts

Algorithm	A generically implemented function that places minimal requirements on the data on which it is executed.
Function Object	An object that behaves as a function. Its advantage compared to a function pointer is that, as an object, it also stores state, not just behaves as a function. Realization: with a class in which we define the function-call operator. Besides this the class can also contain data members and other helper operations.
Sequence	A container in which every element has a fixed position, determined by the place and time of insertion.
Associative Container	A container in which the elements are stored according to some ordering criterion. The place of insertion is determined not by the time of insertion, but by the value of the inserted element.
Function Template	A function equipped with type parameters, which defines a family of functions.
Class Template	A class equipped with type parameters, which defines a family of types.
Virtual Function	A function realizing polymorphic behavior. With the help of the <code>virtual</code> keyword one has to switch on the many-formedness of a given operation.
Pure Virtual Function	A virtual function declaration. An abstract operation whose implementation is not given at the present level. We use it in interface-defining classes and the descendant classes will implement it.
Inline Function	Functions that the compiler expands at the place of the call, that is, no function call happens; instead the code of the function is substituted in place of the call.

Constant Functions	Functions that do not modify the state of the object. For example methods of the <i>getAttribute</i> kind.
Private Inheritance	It has an access-narrowing effect. The data and method members taken over from the ancestor class become private members, so that the descendant class no longer provides the behavior specified by the ancestor class. The descendant class inherits the implementation of the ancestor class and not its interface.
Friend Class	If class A is a friend of class B, then in class A we access the members of class B (private and non-private) in the same way as within class B itself.
Template Specialization	The specialization of a class template for a given type, the goal of which is either optimization for the given type, or avoidance of anomalous behavior that arose for the given type.